

WEEK 9 PRACTICAL

QUESTION 1

Implement a file copy utility using low-level Linux file I/O system calls (open, read, write, lseek, close). Compare its performance with standard library functions (fopen, fread, fwrite).

Aim

To implement a file copy utility in C using the low-level system calls **open()**, **read()**, **write()**, **lseek()** and **close()**, and to compare its performance with the buffered standard I/O library functions **fopen()**, **fread()** and **fwrite()**.

```

18  */
19  #include <stdio.h>
20  #include <stdlib.h>
21  #include <string.h>
22  #include <errno.h>
23  #include <fcntl.h>
24  #include <time.h>
25  #include <unistd.h>
26  #include <sys/stat.h>
27
28  #define TEST_FILE "test_input.bin"
29  #define TEST_SIZE (8 * 1024 * 1024) /* 8 MB test file */
30  #define MAX_BUFFER 8192
31
32  /* ----- Helpers ----- */
33
34  static double elapsed_ms(struct timespec start, struct timespec end)
35  {
36      return (end.tv_sec - start.tv_sec) * 1000.0 +
37             (end.tv_nsec - start.tv_nsec) / 1e6;
38  }
39
40  /* Writes the whole buffer, handling partial writes and interrupts. */
41  static int write_all(int fd, const char *buf, size_t len)
42  {
43      while (len > 0) {
44          ssize_t n = write(fd, buf, len);
45          if (n == -1) {
46              if (errno == EINTR) continue;
47              return -1;
48          }
49          buf += n;
50          len -= (size_t)n;
51      }
52      return 0;
53  }
54
55  /* ----- Copy using system calls ----- */
56
57  static long copy_syscalls(const char *src, const char *dst, size_t bufsize,
58                           long *syscall_count)
59  {
60      char buf[MAX_BUFFER];
61      long total = 0, calls = 0;
62      ssize_t n;
63
64      int in = open(src, O_RDONLY);
65      if (in == -1) { perror(src); return -1; }
66
67      /* lseek to the end gives the file size, then rewind to the start */
68      off_t size = lseek(in, 0, SEEK_END);
69      if (size == -1 || lseek(in, 0, SEEK_SET) == -1) {
70          perror("lseek"); close(in); return -1;
71      }
72
73      int out = open(dst, O_WRONLY | O_CREAT | O_TRUNC, 0644);
74      if (out == -1) { perror(dst); close(in); return -1; }
75
76      while ((n = read(in, buf, bufsize)) > 0) {
77          calls++;
78          if (write_all(out, buf, (size_t)n) == -1) {
79              perror("write"); close(in); close(out); return -1;
80          }
81      }
82
83      close(in); close(out);
84
85      *syscall_count = calls;
86      return total;
87  }
88
89  /* ----- Copy using the standard I/O library ----- */
90
91  static long copy_stdio(const char *src, const char *dst, size_t bufsize)
92  {
93      char buf[MAX_BUFFER];
94      long total = 0;
95      size_t n;
96
97      FILE *in = fopen(src, "rb");
98      if (!in) { perror(src); return -1; }
99      FILE *out = fopen(dst, "wb");
100     if (!out) { perror(dst); fclose(in); return -1; }
101
102     while ((n = fread(buf, 1, bufsize, in)) > 0) {
103         if (fwrite(buf, 1, n, out) != n) {
104             perror("fwrite"); fclose(in); fclose(out); return -1;
105         }
106         total += (long)n;
107     }
108     if (ferror(in)) perror("fread");
109     fclose(in);
110     if (fclose(out) != 0) { perror("fclose"); return -1; }
111     return total;
112 }
113
114 /* ----- Verification: compare two files byte by byte ----- */
115
116 static int files_identical(const char *a, const char *b)
117 {
118     FILE *fa = fopen(a, "rb"), *fb = fopen(b, "rb");
119     int same = (fa && fb);
120     while (same) {
121         int ca = fgetc(fa), cb = fgetc(fb);
122         if (ca != cb) same = 0;
123         if (ca == EOF || cb == EOF) break;
124     }
125     if (fa) fclose(fa);
126     if (fb) fclose(fb);
127     return same;
128 }
129
130 static int create_test_file(void)
131 {
132     char block[4096];
133     int fd = open(TEST_FILE, O_WRONLY | O_CREAT | O_TRUNC, 0644);
134     if (fd == -1) { perror("create test file"); return -1; }
135     for (size_t i = 0; i < sizeof(block); i++)
136         block[i] = (char)('A' + i % 26);
137     for (int i = 0; i < TEST_SIZE / (int)sizeof(block); i++)
138         if (write_all(fd, block, sizeof(block)) == -1) {

```

Program

Output

```

1  /* ----- Performance comparison ----- */
2
3  static void benchmark(void)
4  {
5      const size_t sizes[] = { 16, 4096 };
6      struct timespec t0, t1;
7
8      if (create_test_file() == -1) exit(EXIT_FAILURE);
9      printf("Test file: %s (%d MB)\n", TEST_FILE, TEST_SIZE / (1024 * 1024));
10     printf("%-20s %-8s %-12s %-12s\n",
11           "Method", "Buffer", "Time (ms)", "Syscalls", "Verified");
12     printf("-----\n");
13
14     for (int i = 0; i < 2; i++) {
15         long calls = 0;
16
17         clock_gettime(CLOCK_MONOTONIC, &t0);
18         copy_syscalls(TEST_FILE, "copy_syscall.bin", sizes[i], &calls);
19         clock_gettime(CLOCK_MONOTONIC, &t1);
20         printf("%-20s %-8s %-12.2f %-12s\n",
21               "System calls (read/write)", sizes[i], elapsed_ms(t0, t1), calls,
22               files_identical(TEST_FILE, "copy_syscall.bin") ? "yes" : "NO");
23
24         clock_gettime(CLOCK_MONOTONIC, &t0);
25         copy_stdio(TEST_FILE, "copy_stdio.bin", sizes[i]);
26         clock_gettime(CLOCK_MONOTONIC, &t1);
27         printf("%-20s %-8s %-12.2f %-12s\n",
28               "Standard I/O (fread/fwrite)", sizes[i], elapsed_ms(t0, t1),
29               "buffered",
30               files_identical(TEST_FILE, "copy_stdio.bin") ? "yes" : "NO");
31     }
32
33     unlink(TEST_FILE);
34     unlink("copy_syscall.bin");
35     unlink("copy_stdio.bin");
36 }
37
38 int main(int argc, char *argv[])
39 {
40     if (argc == 3) {
41         long calls = 0;
42         long bytes = copy_syscalls(argv[1], argv[2], 4096, &calls);
43         if (bytes < 0) return EXIT_FAILURE;
44         printf("Copied %ld bytes from %s to %s using %ld read/write calls\n",
45               bytes, argv[1], argv[2], calls);
46         return EXIT_SUCCESS;
47     }
48     if (argc != 1) {
49         fprintf(stderr, "Usage: %s <source> <destination>\n", argv[0]);
50         fprintf(stderr, "    %s (run performance comparison)\n", argv[0]);
51         return EXIT_FAILURE;
52     }
53     benchmark();
54     return EXIT_SUCCESS;
55 }

```

```
chakrika@Chakrikas-MacBook-Air pjsdj % ./w9q1
Test file: test_input.bin (8 MB)
```

Method	Buffer	Time (ms)	Syscalls	Verified
System calls (read/write)	16	1283.62	1048576	yes
Standard I/O (fread/fwrite)	16	47.55	buffered	yes
System calls (read/write)	4096	7.21	4096	yes
Standard I/O (fread/fwrite)	4096	16.01	buffered	yes

```
chakrika@Chakrikas-MacBook-Air pjsdj % █
```

The exact addresses will be different on every execution because of Linux memory-layout randomization.

Observation

The program copied an 8 MB file four ways and verified that every copy is byte-for-byte identical to the original.

Method	Buffer	Behaviour
System calls	16 bytes	Every read() and write() enters the kernel. Over one million system calls were made, so this was by far the slowest method.
Standard I/O	16 bytes	fread()/fwrite() copy into a user-space buffer (typically 4 KB or larger) and call read()/write() only when it fills, so far fewer system calls are made.
System calls	4096 bytes	Each call moves a full block, so only about 4,096 system calls are needed. Performance matches standard I/O.
Standard I/O	4096 bytes	Similar to large-buffer system calls; the extra user-space copy adds very little cost.

- The cost of a copy depends mainly on the **number of system calls**, because each call requires a switch from user mode to kernel mode.
- lseek(fd, 0, SEEK_END) returns the file size, and lseek(fd, 0, SEEK_SET) moves the offset back to the start of the file.

- `write()` may write fewer bytes than requested, so the program loops until the whole buffer is written.
- Standard I/O is faster for small reads and writes because of its internal buffering; with a large buffer, raw system calls perform just as well and give direct control over the file descriptor.

Result

A file copy utility was successfully implemented using the low-level system calls **`open()`**, **`read()`**, **`write()`**, **`lseek()`** and **`close()`**. Its performance was compared with **`fopen()`**, **`fread()`** and **`fwrite()`**, showing that buffering, and therefore the number of system calls, determines file I/O performance.

QUESTION 2

Develop a program that redirects standard input and output using file descriptors and `dup2()`. Demonstrate how shells implement I/O redirection.

Aim:

To redirect standard input, standard output and standard error using file descriptors and the `dup()` and `dup2()` system calls, and to demonstrate how a shell implements the redirection operators `>`, `<`, `2>&1` using `fork()`, `dup2()` and `exec()`.

Program

```
/*
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

static void die(const char *msg)
{
    perror(msg);
    exit(EXIT_FAILURE);
}

static void show_file(const char *path)
{
    char line[256];
    FILE *fp = fopen(path, "r");
    if (!fp) die(path);
    while (fgets(line, sizeof(line), fp))
        printf("    | %s", line);
    fclose(fp);
}

int main(void)
{
    printf("Standard descriptors: stdin = %d, stdout = %d, stderr = %d\n",
        STDIN_FILENO, STDOUT_FILENO, STDERR_FILENO);

    /* ----- Part 1: output redirection (command > output.txt) ----- */
    printf("Part 1: Output redirection -> command > output.txt\n");
    fflush(stdout);

    int saved_stdout = dup(STDOUT_FILENO); /* keep terminal stdout */
    int fd = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (saved_stdout == -1 || fd == -1) die("open/dup");

    dup2(fd, STDOUT_FILENO); /* fd 1 now -> output.txt */
    close(fd);
    printf("This line was written by printf() into output.txt\n");
    printf("printf() still writes to fd 1, but fd 1 now points to the file\n");
    fflush(stdout);

    dup2(saved_stdout, STDOUT_FILENO); /* restore terminal */
    close(saved_stdout);
    printf("    stdout restored. Contents of output.txt:\n");
    show_file("output.txt");

    /* ----- Part 2: input redirection (command < input.txt) ----- */
    printf("\nPart 2: Input redirection -> command < input.txt\n");
    FILE *fp = fopen("input.txt", "w");
    if (!fp) die("input.txt");
    fprintf(fp, "Ravi\nAnita\nKiran\nDeepa\nArjun\n");
    fclose(fp);

    int saved_stdin = dup(STDIN_FILENO);
    fd = open("input.txt", O_RDONLY);
    if (saved_stdin == -1 || fd == -1) die("open/dup");
    dup2(fd, STDIN_FILENO); /* fd 0 now -> input.txt */
    close(fd);
```

```
dup2(fd, STDIN_FILENO); /* fd 0 now -> input.txt */
close(fd);

char name[64];
int count = 0;
while (fgets(name, sizeof(name), stdin)) { /* reads from the file */
    name[strcspn(name, "\n")] = '\0';
    printf("    Read from stdin: %s\n", name);
    count++;
}

dup2(saved_stdin, STDIN_FILENO);
close(saved_stdin);
clearerr(stdin);
printf("    %d names read. stdin restored to the keyboard.\n", count);

/* ----- Part 3: error redirection (command 2>&1 > errors.txt) ----- */
printf("\nPart 3: Error redirection -> command > errors.txt 2>&1\n");
fflush(stdout);
pid_t pid = fork();
if (pid == -1) die("fork");
if (pid == 0) {
    fd = open("errors.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd == -1) die("errors.txt");
    dup2(fd, STDOUT_FILENO); /* > errors.txt */
    dup2(STDOUT_FILENO, STDERR_FILENO); /* 2>&1 */
    close(fd);
    execlp("ls", "ls", "input.txt", "missing_file.txt", (char *)NULL);
    die("execlp");
}

waitpid(pid, NULL, 0);
printf("    Both normal output and error message went to errors.txt\n");
show_file("errors.txt");

/* ----- Part 4: how a shell runs sort < input.txt > sorted.txt ----- */
printf("\nPart 4: Shell simulation -> sort < input.txt > sorted.txt\n");
fflush(stdout);
pid = fork();
if (pid == -1) die("fork");
if (pid == 0) {
    int in = open("input.txt", O_RDONLY);
    int out = open("sorted.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (in == -1 || out == -1) die("open");
    dup2(in, STDIN_FILENO); /* < input.txt */
    dup2(out, STDOUT_FILENO); /* > sorted.txt */
    close(in);
    close(out);
    execlp("sort", "sort", (char *)NULL); /* sort never knows about files */
    die("execlp");
}

int status;
waitpid(pid, &status, 0);
printf("    Child (PID %d) exited with status %d. Contents of sorted.txt:\n",
    pid, WEXITSTATUS(status));
show_file("sorted.txt");

unlink("output.txt");
unlink("input.txt");
unlink("errors.txt");
unlink("sorted.txt");
return EXIT_SUCCESS;
```

```

chakrika@Chakrikas-MacBook-Air pisdj % ./w9q1
Test file: test_input.bin (8 MB)

Method                Buffer   Time (ms)   Syscalls   Verified
-----
System calls (read/write) 16      1283.62    1048576    yes
Standard I/O (rread/fwrite) 16      47.55     buffered    yes
System calls (read/write) 4096     7.21      4096       yes
Standard I/O (fread/fwrite) 4096    16.01     buffered    yes
chakrika@Chakrikas-MacBook-Air pisdj % gcc w9q2.c -o w9q2
chakrika@Chakrikas-MacBook-Air pisdj % ./w9q2
Standard descriptors: stdin = 0, stdout = 1, stderr = 2

Part 1: Output redirection  ->  command < output.txt
      stdout restored. Contents of output.txt:
      | This line was written by printf() into output.txt
      | printf() still writes to fd 1, but fd 1 now points to the file

Part 2: Input redirection   ->  command < input.txt
      Read from stdin: Ravi
      Read from stdin: Anita
      Read from stdin: Kiran
      Read from stdin: Deepa
      Read from stdin: Arjun
      5 names read. stdin restored to the keyboard.

Part 3: Error redirection   ->  command > errors.txt 2>&1
      Both normal output and error message went to errors.txt:
      | ls: missing_file.txt: No such file or directory
      | input.txt

Part 4: Shell simulation     ->  sort < input.txt > sorted.txt
      Child (PID 99387) exited with status 0. Contents of sorted.txt:
      | Anita
      | Anita
      | Kiran
      | Deepa
      | Arjun
chakrika@Chakrikas-MacBook-Air pisdj %

```

Output:

Observation

Every process starts with three open file descriptors: 0 (stdin), 1 (stdout) and 2 (stderr). `dup2(oldfd, newfd)` makes `newfd` refer to the same open file as `oldfd`, so any code that uses `newfd` is silently redirected.

Shell syntax	Equivalent system calls	Effect
<code>cmd > file</code>	<pre>fd = open(file, O_WRONLY O_CREAT O_TRUNC); dup2(fd, 1)</pre>	Standard output is written to the file

Shell syntax	Equivalent system calls	Effect
cmd < file	fd = open(file, O_RDONLY); dup2(fd, 0)	Standard input is read from the file
cmd 2>&1	dup2(1, 2)	Errors go wherever standard output goes
sort < in > out	fork(); in the child: dup2 for fd 0 and fd 1, then execlp("sort")	The shell sets up the descriptors; sort itself is unaware of the files

- `printf()` always writes to descriptor 1. After `dup2(fd, 1)` its output went to `output.txt` without any change to the `printf()` calls.
- Saving the original descriptor with `dup()` allowed the program to restore the terminal afterwards.
- Open file descriptors are inherited across `fork()` and preserved across `exec()`. This is exactly how a shell redirects the I/O of commands such as `sort` and `ls`.
- The unused descriptor is closed after `dup2()` so that no descriptors are leaked.

Result

Standard input, output and error were successfully redirected using **`dup()`** and **`dup2()`**. The program demonstrated how a shell implements output redirection, input redirection, error redirection and combined redirection using **`fork()`**, **`dup2()`** and **`exec()`**.